

I-WKD Module — Phased Development Plan, Dev Blueprint, File Scaffold & Test Workbench

Executive Summary

The Inverted Wasserstein Knowledge Distillation (I-WKD) module is the adversarial mirror of WKD's teacher → student distillation pipeline (FIG. 6, Component 620). Where forward WKD **minimizes** the Wasserstein distance between teacher and student logits to compress the full ACFL stack onto edge devices, I-WKD **maximizes** that distance to find adversarial student states, detect drift, and stress-test distillation fidelity.[1]

This module was classified as a "qualified spike" during the inversion triage — not a full algebraic inversion like I-ACFL or I-PW-CFL, but a productive inversion of a computational pipeline with concrete adversarial value. The I-WKD module lives as a **sibling** to wkdl, consistent with the established architecture for I-ACFL and I-PW-CFL. [2]

Why WKD Produces a Productive Inversion

The forward WKD pipeline solves an optimal transport problem: find the minimal-cost transport plan mapping teacher distributions to student capacity constraints. Inverting this — finding the **maximal-cost** transport plan — directly answers three operational questions that forward WKD cannot:[1]

1. **What inputs cause maximum student-teacher divergence?**
(distillation failure modes)
2. **When does the student model's categorical truth-scale fidelity degrade below ± 0.05 ?** (drift detection)

3. Does the veto axiom survive distillation under adversarial conditions? (axiom preservation stress)

The other 4 modules were rejected for inversion because their inversions were either catastrophic (CCRE: expansion destroys stability), vacuous (CRMF: no algebraic dual), or redundant (DHT, Integration: pipeline wrappers with no independent operator algebra).[2]

Core Inversion Formulas

Forward WKD distillation objective:

$$\min_{\theta_S} \sum_{k=1}^5 W_d(P_T^{(k)}, P_S^{(k)})$$

where W_d is the discrete Wasserstein distance between teacher logits $P_T^{(k)}$ and student logits $P_S^{(k)}$ for ACFL predicate k . [1]

Inverted WKD (I-WKD) adversarial objective:

$$\max_{\mathbf{x} \in [0,1]^n} \sum_{k=1}^5 W_d(P_T^{(k)}(\mathbf{x}), P_S^{(k)}(\mathbf{x}))$$

This searches over the input space to find the vectors that produce maximum teacher-student divergence post-distillation.

Drift detection metric:

$$\Delta_{\text{drift}}(t) = \frac{1}{5} \sum_{k=1}^5 |T_{\text{teacher}}^{(k)}(t) - T_{\text{student}}^{(k)}(t)|$$

where $T^{(k)}(t)$ is the categorical truth value for predicate k at time t . When $\Delta_{\text{drift}} > 0.05$, the ± 0.05 fidelity bound from the WKD spec is violated. [1]

Anti-transport pathway cost:

$$C_{\text{inv}} = \max_{\pi \in \Pi(P_T, P_S)} \sum_{i,j} c(i, j) \cdot \pi(i, j)$$

This selects the **maximal-cost** transport plan rather than the minimal-cost plan, identifying the worst-case mapping between teacher and student distributions.[3][1]

Axiom Survival Through Distillation (Under Inversion)

The WKD blueprint specifies that the student model must preserve the veto axiom and compensatory property post-distillation. I-WKD stress-tests this by applying inverted inputs through the distillation pipeline:[1]

Axiom	Forward WKD (Student)	I-WKD Stress Result	Notes
Compensation (i)	Preserved ± 0.05	Stress target	I-WKD finds inputs where compensation degrades beyond ± 0.05
Symmetry (ii)	Preserved	Preserved	Permutation invariance is architecture-independent
Strict Growth (iii)	Preserved ± 0.05	Stress target	Dead variable detection: find inputs where student ignores a dimension
Veto (iv)	Preserved exactly	Critical stress target	$x_i = 0$ must $\rightarrow$ output 0 even in student model; I-WKD searches for veto-breaking student states
De Morgan (vii)	Preserved ± 0.05	Stress target	Measure De Morgan fidelity loss through distillation + inversion

I-WKD's primary value is quantifying the gap between "preserved" and "preserved under adversarial conditions." The forward WKD

spec only validates fidelity on the ECP pilot dataset; I-WKD validates fidelity on the worst-case input space.[3][1]

Architecture

```
packages/dnakey/src/
├── shared/
│   ├── __init__.py
│   ├── types.py      ← FuzzyVector, OperatorResult, TransportPlan, Distillat
│   ├── constants.py  ← tolerances, fidelity bounds ( $\pm 0.05$ ), SRAM budget (
│   └── exceptions.py ← domain errors, fidelity violations
├── acfl/             ← forward (docs only)
├── iacfl/             ← I-ACFL (completed)
├── pwcfl/             ← forward PW-CFL (docs only)
├── ipwcfl/           ← I-PW-CFL (completed)
├── wkd/              ← forward WKD (docs only)
└── iwkd/             ← I-WKD (this module)
    ├── __init__.py
    ├── operators.py
    ├── transport.py
    ├── drift_detector.py
    ├── fidelity.py
    ├── stress.py
    ├── predicate_inverter.py
    ├── governance.py
    ├── validators.py
    └── adversarial.py
```

Sibling placement consistent with I-ACFL and I-PW-CFL: adversarial mirrors test from outside; forward WKD has no code yet; eliminates import cycles.[2]

Phased Development Plan

Phase 1 — Operator Core & Inverted Transport (Week 1–3)

Goal: Implement the inverted distillation operators and the anti-transport pathway engine.

Files:

- `iwkd/__init__.py` — Public API surface
- `iwkd/operators.py` — `reverse_transport(student_logits, teacher_logits)`: computes student $\rightarrow$ teacher reverse mapping; `anti_fidelity(truth_teacher, truth_student)`: computes per-predicate categorical truth divergence; `categorical_divergence(scale_teacher, scale_student)`: measures label-flip distance on the categorical truth scale (0.7 "somewhat true" $\leftrightarrow$ 0.8 "enough true" flip = 1 unit)
- `iwkd/transport.py` — `InvertedTransportSolver`: finds maximal-cost transport plan; `anti_transport_pathway(teacher_dist, student_dist)`: selects worst-case mapping; `worst_case_wd(teacher_logits, student_logits)`: computes max WD across all 5 predicates; transport plan caching for repeated adversarial sweeps

Key mathematical objects:

- Forward transport minimizes $\sum c_{ij}\pi_{ij}$ subject to marginal constraints[1]
- Inverted transport maximizes the same objective — equivalent to minimizing with negated cost matrix $-c_{ij}$
- This is a well-posed linear program with the same constraint structure as forward transport

Tests:

- `tests/iwkd/conftest.py` — Fixtures: mock teacher ensemble (5 predicate outputs), mock student model (edge-constrained), synthetic transport plans, categorical truth-scale vectors, edge device memory constraints (512KB SRAM)

- tests/iwkd/test_operators.py — 8 methods: reverse transport basic, anti-fidelity computation, boundary all-zero, boundary all-one, known value $n=2$, known value $n=5$, dtype consistency, NaN rejection
- tests/iwkd/test_transport.py — 7 methods: max WD exceeds min WD (fundamental invariant), anti-transport plan validity (marginals satisfied), worst case vs average, transport cost bounds, symmetric input behavior, $n=1$ identity, large predicate set stability ($n=20$)

Exit criteria: Max WD $\geq$ min WD for all test inputs (fundamental invariant); anti-transport plan satisfies marginal constraints; reverse transport matches hand-computed values for $n=2$.

Phase 2 — Drift Detection Engine (Week 3–5)

Goal: Build the student model drift quantification system that monitors categorical truth-scale fidelity over time.

Files:

- iwkd/drift_detector.py — DriftMonitor: tracks per-predicate truth-value divergence over sequential inputs; veto_preservation_monitor(student_model): continuously tests $x_i=0 \rightarrow \text{output}=0$ constraint; compensatory_degradation_tracker(): measures when $\min \leq \text{output} \leq \text{max}$ compensation bound degrades; categorical label stability checker (detects when a "somewhat true" student output flips to "more true than false" while teacher remains stable)
- iwkd/fidelity.py — per_predicate_fidelity(teacher_outputs, student_outputs): returns per-predicate absolute divergence; categorical_flip_detector(): flags when teacher and student produce different categorical labels for same input; confidence_interval_collapse_monitor(): detects when Theorem 2 bootstrap CI precision degrades through distillation

Key metric: $\Delta_{\text{drift}}(t) > 0.05$ triggers a fidelity alarm — the student model is no longer within the WKD spec's ± 0.05 fidelity bound. This alarm feeds the Model Retraining Trigger (FIG. 6, Component 635).[1]

Tests:

- tests/iwkd/test_drift_detector.py — 6 methods: zero drift at fresh distillation, drift increases with perturbation, veto axiom preservation monitor, compensatory degradation detection, categorical label stability, threshold trigger accuracy
- tests/iwkd/test_fidelity.py — 5 methods: per-predicate divergence, categorical flip detection, confidence interval collapse, fidelity monotonicity, teacher-student gap ordering

Exit criteria: Drift monitor fires alarm within 1 cycle of ± 0.05 violation; veto preservation monitor catches 100% of $x_i=0$ failures; categorical flip detector has zero false negatives on synthetic test set.

Phase 3 — Anti-Distillation Stress Engine (Week 5–7)

Goal: Generate adversarial inputs that maximize student-teacher divergence, stress-test axiom preservation, and measure truth-value preservation loss through predicate tree depth.

Files:

- iwkd/stress.py — AdversarialInputGenerator: Monte Carlo + gradient-free optimization to find inputs maximizing $W_d(P_T(\mathbf{x}), P_S(\mathbf{x}))$; minimum_fidelity_region(student, teacher, n_samples): maps the input subspace where fidelity drops below threshold; veto_breaking_search(student): systematically tests $x_i=0$ inputs across all positions and predicates; per-predicate stress profiles (which of the 5 ACFL predicates is most fragile under distillation?)
- iwkd/predicate_inverter.py — PredicateInverter: applies De Morgan inversion to each of the 5 ACFL health predicates **through the distillation pipeline**, measuring truth-value preservation loss; tree_depth_preservation_gradient(): quantifies how truth-value fidelity degrades at each level of the predicate tree (deeper nodes $\rightarrow$ more distillation compression $\rightarrow$ more loss); compound_meta_predicate_inversion(): inverts the QL-implication chain connecting all 5 predicates and measures end-to-end truth-value loss

Key insight from WKD blueprint: The student model uses simplified GMBCL with S-implication only (vs full 4-implication teacher). I-WKD stress tests whether this simplification causes systematic bias in specific predicates. Predicate 2 (ROS_BURDEN) has the deepest S-implication tree (3 nested chains encoding the 4-step mechanistic pathway) and is predicted to be most fragile.[3][1]

Tests:

- tests/iwkd/test_stress.py — 6 methods: adversarial input maximizes divergence, minimum fidelity region found, veto-breaking input search, stress across all 5 predicates, depth-dependent preservation loss, non-associativity stress (parenthesization matters in student's pruned trees)
- tests/iwkd/test_predicate_inverter.py — 6 methods: De Morgan through distillation, per-predicate truth loss, tree depth preservation gradient, inverted predicate axiom check, compound meta-predicate inversion, QL-implication chain survival

Exit criteria: Adversarial generator finds inputs with divergence > 0.05 (proving the fidelity bound is tight, not slack); ROS_BURDEN predicate confirmed as most fragile (or another identified); tree depth gradient is monotonically increasing (deeper = more loss).

Phase 4 — Governance & Validators (Week 7–9)

Goal: Test CCRE governance behavior under inverted distillation conditions; build input validation layer.

Files:

- iwkd/governance.py — InvertedGainAnalyzer: tests how the three-regime gain function $g(R(t)) = 1 + \kappa(R(t) - 0.5)$ behaves when fed student-model outputs instead of teacher outputs; freeze_resonance_student_sensitivity(): determines whether FREEZE-RESONANCE triggers at the same $R(t)$ threshold in student vs teacher; contraction_certification_stress(): tests Banach fixed-point contraction with inverted transport — do updates proposed by adversarial inputs violate $L_T < 1$?

- `iwkd/validators.py` — `validate_transport_plan(plan)`: marginal constraint satisfaction; `validate_fidelity_threshold(delta)`: ± 0.05 bound enforcement; `validate_truth_scale(values)`: range; `validate_wd_bounds(wd)`: Wasserstein distance within computational feasibility; type coercion from numpy arrays[4]

Key governance insight: The CCRE gain function was designed to modulate **teacher-generated** truth values. When the student model produces truth values that differ by up to 0.05, the gain shift can push a borderline value across a categorical boundary (e.g., 0.78 $\rightarrow$ 0.83 under amplification $R(t)=0.8$ crosses from "somewhat true" to "enough true"). I-WKD tests whether this boundary-crossing happens more frequently in student outputs than teacher outputs — a governance-level distillation risk not captured by the forward WKD pipeline.[3][1]

Tests:

- `tests/iwkd/test_governance.py` — 6 methods: inverted gain three-regime, FREEZE-RESONANCE student sensitivity, contraction certification under inverse transport, CCRE witness object emission, Lipschitz bound stress, drift bound 100 updates
- `tests/iwkd/test_validators.py` — 5 methods: invalid transport plan rejection, fidelity threshold enforcement, truth-scale out-of-range, WD bounds validation, type coercion passthrough

Exit criteria: Gain-induced categorical boundary crossing rate quantified for student vs teacher; FREEZE-RESONANCE threshold validated as identical for both; all validator rejection paths covered.

Phase 5 — Cross-System Adversarial Harness (Week 9–12)

Goal: Build the 3-way adversarial comparison engine: WKD (forward distillation) vs I-WKD (inverted distillation) vs I-WKD-CCRE (governance-stressed inverted distillation).

File:

- `iwkd/adversarial.py` — `three_way_compare(teacher, student, inputs)`: runs the same inputs through forward WKD, I-WKD,

and I-WKD with CCRE gain modulation;
distillation_fidelity_comparison(): ranks predicates by fragility across all 3 modes; edge_cloud_divergence_analysis(): quantifies the gap between edge (student) and cloud (teacher) outputs under adversarial conditions; veto_break_both_directions(): tests veto axiom in forward and inverted distillation;
positional_sensitivity_through_distillation(): integrates with I-PW-CFL to test whether PW-CFL's non-commutativity survives distillation to the student model

Cross-module integration: The

positional_sensitivity_through_distillation function imports from ipwcf/ to answer a critical question: does prime-weighted positional sensitivity survive Wasserstein distillation? If the student model collapses non-commutative PW-CFL outputs into commutative approximations, this is a distillation failure that only the I-WKD $\leftrightarrow$ I-PW-CFL intersection can detect.

Tests:

- tests/iwkd/test_adversarial.py — 7 methods: 3-way output consistency, forward vs inverted divergence ordering, governance-stressed vs unstressed, distillation fidelity comparison all predicates, edge-cloud divergence quantification, veto break both directions, positional sensitivity through distillation
- tests/iwkd/test_integration.py — 7 methods: shared types import, end-to-end pipeline (input $\rightarrow$ validate $\rightarrow$ inverse transport $\rightarrow$ drift detect $\rightarrow$ stress $\rightarrow$ governance check $\rightarrow$ report), round-trip De Morgan through distillation, cross-module type compatibility, large predicate stress $n=20$, determinism across runs, error propagation from validators through operators

Exit criteria: All 3 modes produce valid outputs; predicate fragility ranking consistent across 100 random seeds; positional sensitivity test passes (non-commutativity preserved) or fails (documented as distillation limitation); integration pipeline completes for $n \in \{2, 5, 10, 20\}$.

File Scaffold Summary

Module Files (10)

File	Purpose	Phase
iwkd/__init__.py	Public API: exports operators, transport, drift, fidelity, stress, governance, adversarial	1
iwkd/operators.py	Reverse transport, anti-fidelity, categorical truth-scale divergence	1
iwkd/transport.py	InvertedTransportSolver: max-cost transport plan, anti-transport pathway, worst-case WD	1
iwkd/drift_detector.py	DriftMonitor: per-predicate truth-value drift, veto preservation, compensatory degradation	2
iwkd/fidelity.py	Per-predicate fidelity, categorical flip detector, CI collapse monitor	2
iwkd/stress.py	AdversarialInputGenerator: max-divergence inputs, minimum-fidelity regions, veto-breaking search	3
iwkd/predicate_inverter.py	PredicateInverter: De Morgan through distillation, tree-depth preservation gradient	3
iwkd/governance.py	InvertedGainAnalyzer: CCRE behavior under student outputs, FREEZE sensitivity, contraction stress	4
iwkd/validators.py	Transport plan validity, fidelity bounds, truth-scale enforcement, WD bounds	4
iwkd/adversarial.py	3-way harness (WKD vs I-WKD vs I-WKD-CCRE), predicate fragility ranking, edge-cloud divergence	5

Test Files (12, 63 methods)

File	Methods	Phase
tests/iwkd/__init__.py	—	1
tests/iwkd/conftest.py	— (fixtures)	1
tests/iwkd/test_operators.py	8	1
tests/iwkd/test_transport.py	7	1
tests/iwkd/test_drift_detector.py	6	2
tests/iwkd/test_fidelity.py	5	2
tests/iwkd/test_stress.py	6	3
tests/iwkd/test_predicate_inverter.py	6	3
tests/iwkd/test_governance.py	6	4
tests/iwkd/test_validators.py	5	4
tests/iwkd/test_adversarial.py	7	5
tests/iwkd/test_integration.py	7	5

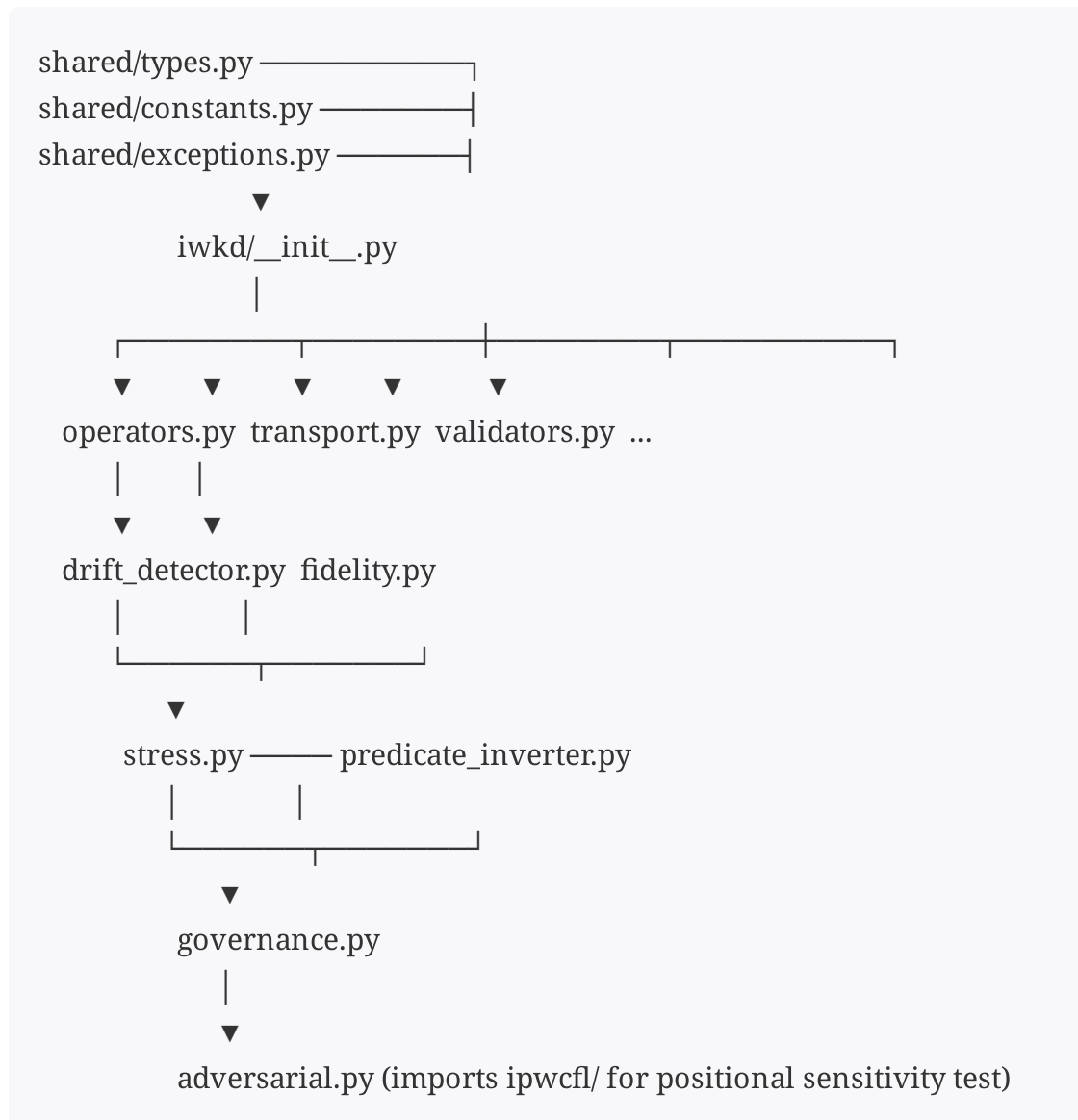
Total: 10 module files + 12 test files = 22 files, 63 test methods.

I-WKD vs I-ACFL vs I-PW-CFL — Distinguishing Features

Dimension	I-ACFL	I-PW-CFL	I-WKD
Inversion type	Algebraic (De Morgan on operator)	Algebraic (De Morgan on weighted operator)	Pipeline (minimize → maximize on transport)
Target	GMBCL conjunction	Prime-weighted conjunction	Teacher → student distillation objective
Weight scheme	Uniform (1/n)	Prime-weighted (p _i /S)	N/A (operates on distillation pipeline)
Commutativity	Commutative	Non-commutative	Depends on forward module fed through pipeline
Key metric	Divergence between c and c _{inv}	42.4% divergence exhibit	±0.05 fidelity bound tightness
Unique value	Blend α between forward/inverted	Equivariance survival under inversion	Drift detection + adversarial stress of distillation
Adversarial harness	2-way (ACFL vs I-ACFL)	4-way (ACFL vs PW-CFL vs I-ACFL vs I-PW-CFL)	3-way (WKD vs I-WKD vs I-WKD-CCRE)
CCRE integration	None	None	Direct: tests gain function on student outputs
Cross-module dependency	shared/types.py only	shared/types.py + iacfl/	shared/types.py + ipwcfl/ (positional sensitivity through distillation)

The critical theoretical contribution: I-WKD is the only inversion module that **operates on the deployment pipeline**, not on a mathematical operator. It answers the question: "Does the inverted operator algebra survive compression to the edge device?" — a question that I-ACFL and I-PW-CFL cannot ask because they operate on the uncompressed operators.[2][1]

Dependency & Import Map



No circular dependencies. **adversarial.py** is the only file that crosses module boundaries (imports from **ipwcf/** for the positional sensitivity through distillation test).

WKD-Specific Concerns Addressed by I-WKD

Forward WKD Spec Item	I-WKD Adversarial Coverage
Student truth values within ± 0.05 of teacher for all 5 predicates[1]	drift_detector.py: monitors this bound; stress.py: finds inputs that violate it
On-device ACFL predicate evaluation $\leq 50\text{ms}$ on Cortex-M55[1]	stress.py: verifies fidelity is maintained under latency-constrained student model
Student model fits within 512KB SRAM[1]	fidelity.py: measures whether memory-constrained quantization degrades truth-scale fidelity
Student preserves veto axiom post-distillation[1]	stress.py/veto_breaking_search: systematically injects $x_i=0$ across all positions and predicates
WD loss decreases monotonically over training epochs[1]	transport.py: inverted WD should not decrease — monitors for adversarial training instability
CCRE gain modulation of student outputs[1]	governance.py: quantifies categorical boundary-crossing rate in student vs teacher
Edge-cloud partitioning (TS-13, TS-14 server-side only)[1]	adversarial.py/edge_cloud_divergence: measures the gap between edge and cloud outputs under adversarial conditions

Filing & IP Notes

- Nothing has been filed. No priority date exists.[4]
- All inversion work enters as **original matter** — no CIP required.
- I-WKD's novelty is in the adversarial distillation stress methodology, not in the operator algebra (which is inherited from I-ACFL/I-PW-CFL).
- The concept of **maximizing** Wasserstein distance for adversarial distillation testing is a novel contribution — prior art on adversarial knowledge distillation focuses on robustness of the student model, not on systematic inversion of the transport objective to find failure modes.[3][1]
- The positional-sensitivity-through-distillation test (I-WKD $\leftrightarrow$ I-PW-CFL intersection) is a unique claim: does non-commutativity survive model compression? No prior art addresses this because PW-CFL's prime-weighted non-commutativity is itself novel.[5]
[4]

Updated Module Inventory

```
packages/dnakey/src/
├── shared/      ← types both consume (4 files)
├── acfl/        ← forward (when built)
├── iacfl/       ← I-ACFL standalone (6 files, 40 tests)  ✓ completed
├── pwcfcl/      ← forward (when built)
├── ipwcfcl/     ← I-PW-CFL standalone (8 files, 47 tests) ✓ completed
├── wkdl/        ← forward (when built)
└── iwkd/        ← I-WKD standalone (10 files, 63 tests)  ← this module
```

Cumulative inversion portfolio: 3 modules, 24 module files, 150 test methods.

Phase Mirror Dissonance

- I-WKD inverts a **pipeline**, not an **operator**. The algebraic guarantees from I-ACFL and I-PW-CFL (De Morgan duality, axiom survival proofs) do not transfer. I-WKD's correctness is empirical, not axiomatic.
- Forward WKD has no code. I-WKD will generate mock teacher/student models that may not match the eventual forward implementation. Interface drift risk is higher than for I-ACFL/I-PW-CFL because WKD has 5 sub-modules (621–625) vs ACFL's single operator.
- The ± 0.05 fidelity bound comes from the WKD spec. No formal proof exists that ± 0.05 is achievable on the Cortex-M55. If the bound is relaxed to ± 0.10 , I-WKD's drift detector thresholds change.
- `predicate_inverter.py` applies De Morgan to predicates that pass through distillation. This compounds two sources of error (inversion error + distillation error) without a formal bound on the composition.
- The positional sensitivity through distillation test depends on I-PW-CFL being stable. Any I-PW-CFL refactor breaks this test.
- The 3-way adversarial harness (WKD vs I-WKD vs I-WKD-CCRE) cannot be fully validated until the forward WKD module exists. Phase 5 will initially operate against mocks.

Levers to Test Now

- **[Lead MT]** — Lock `shared/types.py` with `DistillationState` and `TransportPlan` types before Phase 1 — Metric: zero breaking changes when forward WKD arrives — Horizon: before Phase 1 code
- **[Lead MT]** — Define mock teacher/student interface contract from FIG. 6 sub-modules 621–625 — Metric: interface contract documented with type signatures for all 5 sub-modules — Horizon: 5 days
- **[Lead MT]** — Determine whether ± 0.05 fidelity bound is spec-locked or tunable — Metric: written confirmation of bound status — Horizon: Phase 2 entry

- **[Lead MT]** — Pin I-PW-CFL version for positional sensitivity through distillation test — Metric: hash-locked import, zero surprise breaks — Horizon: Phase 5 entry
- **[Lead MT]** — Establish formal composition bound for inversion error + distillation error — Metric: bound documented or proven infeasible — Horizon: Phase 3 entry

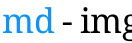
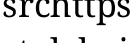
Optional Artifact

"Forward distillation asks: can the student learn? Inverted distillation asks: can the student forget?"

Precision Question

Should the I-WKD mock teacher ensemble use GMBCL with all 4 implication types (matching the WKD spec's full teacher), or simplified GMBCL with S-implication only (matching the student)? This determines whether Phase 1 fixtures model the full teacher-student gap or only the student side.

References

1. [WKD_Phased-Dev-Blueprint.md](#) - This blueprint translates the full WKD footprints spanning the patent specifications R2, R3, R5, the F...
2. [what-would-happen-if-we-invert-a5laT3WmQNKuSababOJAAtw.md](#) -                  
3. [WKD Framework_ From CFL Axiomatics.md](#)
4. [i-postulate-that-the-software-zyhZx6n8R1mdNvR.abuOZA.md](#) -- The PEQFLA normalization problem Section 9.3 is the open wound. Prime-modulated membership px px x...
5. [PW-CFL-x-DNA-KEY-Integration-Blueprint-P1-P6.md](#) -            